

ID	Misconception
CO-1	Using a literal value (e.g., 3.14) as if it were a variable and expecting to assign or change it.
CO-2	Confusing named constants with variables and believing a constant can be reassigned.
CO-3	Failing to distinguish constants from values computed at runtime.
ID	Misconception
VA-1	A variable can hold more than one value at the same time.
VA-2	Assignment statements are commutative ($A = B$ is equivalent to $B = A$).
VA-3	$A = B + 1$ stores the formula rather than the computed value.
VA-4	Assigning one variable to another creates a permanent link between them.
VA-5	Variable names determine their values automatically.
VA-6	Mixing incompatible data types is always safe.
VA-7	Variable names are case-insensitive.
VA-8	Variables behave like mathematical unknowns rather than storage locations.
VA-9	Variable scope is not understood (local vs global confusion).
VA-10	The left-hand side of an assignment does not need to be a variable.
ID	Misconception
OP-1	Using $=$ instead of $==$ for comparison.
OP-2	Boolean operators work exactly like natural language.
OP-3	OR is exclusive (XOR behavior).
OP-4	Integer division always produces decimals.
OP-5	Incorrect assumptions about operator precedence.
OP-6	$= +$ always means arithmetic addition.
ID	Misconception
EX-1	Expressions are stored as formulas instead of evaluated values.
EX-2	Programming expressions behave exactly like algebra.
EX-3	Entire statements are stored in memory rather than computed results.
EX-4	Short-circuit evaluation is not understood.

EX-5	Variables have unlimited size and precision.
EX-6	Variables changed inside loops always produce the same output.
ID	Misconception
IO-1	Input always produces numeric values.
IO-2	Printing a variable changes its value.
IO-3	INPUT A inputs the letter "A" instead of a value stored in A.
IO-4	Output happens automatically whenever a variable changes.
IO-5	Not understanding where input data comes from or where it is stored.
ID	Misconception
CD-1	Both if and else branches always execute.
CD-2	A false condition halts the entire program.
CD-3	if waits until its condition becomes true.
CD-4	The then branch always executes regardless of the condition.
CD-5	A branch executes even after a false condition.
CD-6	Order of conditions in if-elif statements does not matter.
CD-7	Branch execution is unrelated to condition truth values.
CD-8	Conditions are checked again after the branch executes.
CD-9	Code after an if-statement is considered inside a branch.
CD-10	Selection exits before checking all conditions.
CD-11	The same condition is evaluated multiple times unnecessarily.
CD-12	A wrong branch executes after a true condition.
CD-13	Selection statements are iterative like loops.
ID	Misconception
LO-1	First loop iteration occurs before checking the condition.
LO-2	Loop continues without rechecking the condition.
LO-3	Loop continues even when the condition becomes false.
LO-4	Code after the loop is considered part of the loop.

LO-5	Loop body execution can be skipped despite a true condition.
LO-6	for loop initialization runs every iteration.
LO-7	Loop update step is skipped after iterations.
LO-8	for loop initialization is skipped entirely.
LO-9	Only the output statement repeats inside a loop.
LO-10	While-loop conditions are checked at the end (do-while behavior).
LO-11	Indentation does not matter inside loops.
LO-12	Outer and inner loops execute the same number of times.
LO-13	One loop type is always better than another.
LO-14	repeat(N) repeats N+1 times.
LO-15	Loop output remains the same every iteration.
LO-16	Statements inside a loop repeat separately instead of as a group.
LO-17	Block scope inside loops is not understood.